

TimeGen AI

Complete File, Folder and Technology Explanation

A teacher-ready guide to explain what was used, why it was used, and what every project file does.

Project area	Meaning
Frontend	The screens and controls students/teachers/administrators use in the browser.
Backend	The server that handles requests, timetable generation, AI calls, and database work.
Database	MongoDB Atlas storage, with a local JSON fallback if Atlas is unavailable.
AI	Google Gemini model used for timetable assistance and conflict explanations.

Prepared from the current project files on 2026-08-13.

1. What the Project Does

TimeGen AI is an automatic college timetable generator. An administrator first enters teachers, classes, subjects, rooms, labs, subject assignments, timing rules, breaks, and unavailable periods. The scheduler then builds a weekly timetable that avoids clashes and gives special treatment to lab practicals.

The project is a full-stack web application: React renders the application in the browser; an Express server provides APIs; MongoDB Atlas persists the data; and Google Gemini gives natural-language support. All main code is TypeScript, which makes data formats safer and easier to maintain.

2. Technologies Used

Technology	Why it is used in this project
React 19 + React DOM	Builds reusable frontend pages and interactive components.
TypeScript	Adds types for teachers, subjects, entries, API responses and more.
Vite	Fast frontend development server and production bundler.
Express	Node.js backend framework for REST API endpoints.
MongoDB Node Driver	Connects application data to MongoDB Atlas or local MongoDB.
MongoDB Atlas	Cloud-hosted database cluster used for permanent data storage.
dotenv	Reads secrets such as MONGODB_URI and GEMINI_API_KEY from .env.
@google/genai	Official JavaScript SDK used to call Google Gemini.
Gemini 3.6 Flash	LLM configured in this project for chat and conflict explanations.
Tailwind CSS	Utility-first CSS styling for layouts, colours, responsive design.
Lucide React	Professional icons for menus, forms and action buttons.
Motion	UI animation library.
Recharts	Charts used on analytics/dashboard pages.
tsx / esbuild	Runs TypeScript server in development and bundles it for production.

3. Folder Structure

Folder	What it means / what to tell the teacher
src/	All frontend source code. This is the React user interface.
src/components/	Small reusable UI parts used by more than one page.
src/pages/	Each file represents a full screen or feature in the application.
src/services/	Frontend API communication code. It sends requests to the Express backend.
backend/	Server-side feature code: database storage, schedule logic, demo data and AI.
backend/database/	Abstraction for MongoDB Atlas and local JSON storage.
backend/scheduler/	The actual timetable allocation and conflict-checking algorithms.
backend/seed/	Initial sample data used to demonstrate the project.
backend/ai/	Google Gemini integration.
assets/	Reserved for project assets. Currently only contains an AI Studio ignore rule.
node_modules/	Downloaded packages. Automatically created by npm install; not handwritten source code.
output/pdf/	Generated PDF documentation. Safe to keep or share.
tmp/pdfs/	Temporary PDF creation and preview files. Safe to delete.

4. Root Files

File	What it does
server.ts	Main backend entry point. Starts Express on port 3000, defines API routes, initializes database, runs seeding, calls solver/validator/AI, and hosts Vite in development.
package.json	Lists project dependencies and scripts: npm run dev, build, start and lint.
package-lock.json	npm lock file; fixes exact installed package versions for consistent setup.
bun.lock	Bun package-manager lock file. It is not needed if only npm is used, but can be kept.
index.html	Minimal HTML document in which React is mounted.
vite.config.ts	Vite configuration: React and Tailwind plugins, @ path alias, hot reload settings.
tsconfig.json	TypeScript compiler rules.
README.md	Basic project introduction, requirements and commands.
metadata.json	Metadata from the project generation environment.
.gitignore	Prevents secrets, logs, node_modules and build artifacts from being saved to Git.
.env	Private configuration: Atlas URI, Gemini key and optional APP_URL. Never share or commit it.
.env.example	Safe example/template of required environment variables.
.env.local	Extra local environment file. The server currently loads .env.
timegen_local_db.json	Local fallback data store. Used only if MongoDB connection is missing or fails.

5. Frontend Entry and Shared Files

Path	Detailed responsibility
src/main.tsx	Browser entry point. Finds the HTML root element and mounts the React application.
src/App.tsx	Main application container. Chooses which page appears and holds app-level layout/navigation behaviour.
src/index.css	Global visual styles and Tailwind CSS imports/utilities.
src/types.ts	Shared TypeScript data contracts. Defines Classroom, Lab, Batch, StudentClass, Teacher, Subject, ClassSubject, TimingConfig, AvailabilityRule, TimetableEntry, reports and AI messages. It is the common language between UI and backend.
src/services/api.ts	Single typed API client. All pages use it to fetch/create/update/delete data through /api routes instead of writing fetch logic repeatedly.
src/components/Navigation.tsx	Sidebar/navigation menu that lets users switch application sections.
src/components/AIChatDrawer.tsx	Slide-out UI for asking timetable questions in natural language.
src/components/ConfirmModal.tsx	Reusable warning dialog before delete/clear actions.

6. Frontend Pages - Master Data

Path	What the screen lets the user do
src/pages/DashboardPage.tsx	Shows summary counts, quick status, and high-level project information.
src/pages/TeachersPage.tsx	Add, edit, search and delete faculty records including department and daily workload limit.
src/pages/SubjectsPage.tsx	Maintain the subject master list: name, subject code, theory/practical type and description.
src/pages/ClassroomsPage.tsx	Maintain normal teaching rooms: room number, capacity, building and floor.
src/pages/LabsPage.tsx	Maintain laboratories: lab code, lab type, bench capacity, building and floor. Bench capacity is used to split classes into batches.
src/pages/ClassesPage.tsx	Maintain student classes, student counts and batches. Also supports subject assignment at the class level.
src/pages/AssignmentsPage.tsx	Assigns a subject and teacher to a class. Defines weekly theory and practical hours, lab venue, and batch requirement. Practical hours are set in 2-hour increments.

7. Frontend Pages - Timetable Features

Path	What the screen lets the user do
src/pages/TimingsPage.tsx	Sets working days, college start/end time, period length and user-defined breaks. It displays one daily preview of the generated time slots.
src/pages/AvailabilityPage.tsx	A teacher/room/lab/class availability matrix. User clicks a period to block or unblock it. The page uses one row per period, avoiding duplicate rows.
src/pages/GeneratePage.tsx	Triggers automatic schedule generation and presents the run result.
src/pages/TimetablePage.tsx	Displays the weekly timetable by class, teacher, room or lab. Supports edit, delete, lock/unlock and print. Two-hour labs are centred across their two rows. Batch A has blue blocks and Batch B has orange blocks for easier reading.
src/pages/ConflictsPage.tsx	Shows validation problems and uses Gemini to help explain them.
src/pages/AnalyticsPage.tsx	Displays timetable statistics, theory/practical count and visual charts.
src/pages/ExportPage.tsx	Exports timetable information, such as CSV data, for use outside the application.
src/pages/SettingsPage.tsx	Administrative maintenance, including seed demo data and clear/reset data actions.

8. Backend Files

Path	Detailed responsibility
backend/database/store.ts	Database layer. First tries MONGODB_URI. If successful, it uses MongoDB collections. If unavailable, it reads/writes timegen_local_db.json. It exposes the same methods for classrooms, labs, classes, teachers, subjects, assignments, timings, availability, timetable entries and generation runs.
backend/scheduler/solver.ts	Creates a timetable. Generates period slots from timing configuration, excludes breaks, creates batches using lab capacity, builds theory/practical activities, finds suitable rooms/labs, respects availability and prevents overlapping resources. Labs use 2 consecutive periods.
backend/scheduler/validator.ts	Checks the generated/manual timetable for teacher clashes, resource clashes, class clashes, batch clashes, capacity problems, blocked periods and teacher workload issues. It returns conflicts, warnings and a quality score.
backend/seed/demoData.ts	Creates realistic demonstration data: teachers, rooms, labs, classes, subjects, assignments, timing configuration and a first timetable.
backend/ai/assistant.ts	Initializes Google Gen AI, gathers selected live database context, calls Gemini 3.6 Flash for chat, and requests JSON-formatted conflict suggestions. Has fallback responses when AI is unavailable.

9. Database Explanation

The application is configured to prefer MongoDB Atlas. MongoDB Atlas is a cloud database service. The connection string is stored in .env as MONGODB_URI, and the selected database is timegen_ai. If Atlas cannot authenticate or connect, the application does not stop; it instead uses the local file timegen_local_db.json.

MongoDB collection	Information stored
classrooms	Normal room records.
labs	Laboratory records and bench capacity.
student_classes	Class records and their generated batch details.
teachers	Faculty records and workload rules.
subjects	Subject master records.
class_subjects	Class-to-subject-to-teacher assignments, hours, lab and batch requirements.
time_slots	One timing configuration document including work days and breaks.
resource_availability	Periods blocked for teachers, rooms, labs or classes.
timetable_entries	Generated/editable timetable sessions, assigned resource, lock state and duration.
generation_runs	Each generation result: quality score, warnings, conflicts and time taken.

Data safety statement

Do not put passwords, MongoDB URIs, or Gemini API keys in a report, screenshot, GitHub repository, or WhatsApp message. Use .env locally and rotate any secret that has been exposed.

10. AI / LLM Explanation

The LLM used is **Gemini 3.6 Flash**, called through the **@google/genai** SDK. The AI does not generate the actual core schedule allocation; the deterministic TypeScript solver does that. Gemini is used as an assistant: it answers questions about the schedule, describes resources, and explains conflicts in simple language.

11. How the Timetable Algorithm Works

Step	What happens
1. Read input	Server reads resources, classes, teachers, subjects, assignments, timing configuration and availability.
2. Create slots	solver.ts converts start/end time and breaks into teaching periods for each working day.
3. Create batches	If a class has more students than a lab capacity, it is split into Batch A, Batch B and so on.
4. Create activities	Theory hours become one-period theory activities. Practical hours 2/4/6 become 1/2/3 two-period laboratory sessions.
5. Find a slot	The solver searches non-break, non-blocked periods and suitable available rooms/labs.
6. Prevent clashes	It rejects a choice if the teacher, class, batch, room or lab is already occupied in the same period.
7. Save entries	Successful placements are stored as timetable_entries. Locked entries are preserved.
8. Validate	validator.ts checks the final result again and calculates quality/conflicts.

12. Running and Explaining the Project

Command / item	Explanation
npm install	Downloads packages into node_modules.
npm run dev	Runs tsx server.ts. Express starts on port 3000 and includes Vite frontend middleware.
http://localhost:3000	Local browser URL used to operate the application.
npm run lint	Runs TypeScript type checking without changing code.
npm run build	Builds frontend and bundles backend for production.
npm run start	Runs the compiled production backend from dist/server.cjs.
MongoDB Atlas Data Explorer	Web interface where teacher can view timegen_ai collections and documents.

Short teacher presentation script

"TimeGen AI is a full-stack college timetable generator. We built the interface in React and TypeScript, the APIs in Express/Node.js, and use MongoDB Atlas for cloud data storage. The core timetable allocation uses our own TypeScript scheduling algorithm, which avoids teacher, lab, classroom, class and batch conflicts. Google Gemini 3.6 Flash is added as an assistant to explain conflicts and answer timetable questions. Laboratories are scheduled in 2-hour blocks, and Batch A and Batch B use different colours so students and teachers can understand the schedule quickly."

13. What Can Be Deleted

Item	Can delete?	Effect
node_modules/	Yes	npm install will recreate it; required to run the app afterward.
tmp/	Yes	Temporary PDF creation/render files only.
output/	Optional	Removes generated documentation PDFs.
timegen_local_db.json	Only if intended	Erases local fallback data, not MongoDB Atlas data.
src/, backend/, server.ts, package.json, .env	No	These are source/configuration required for the application.